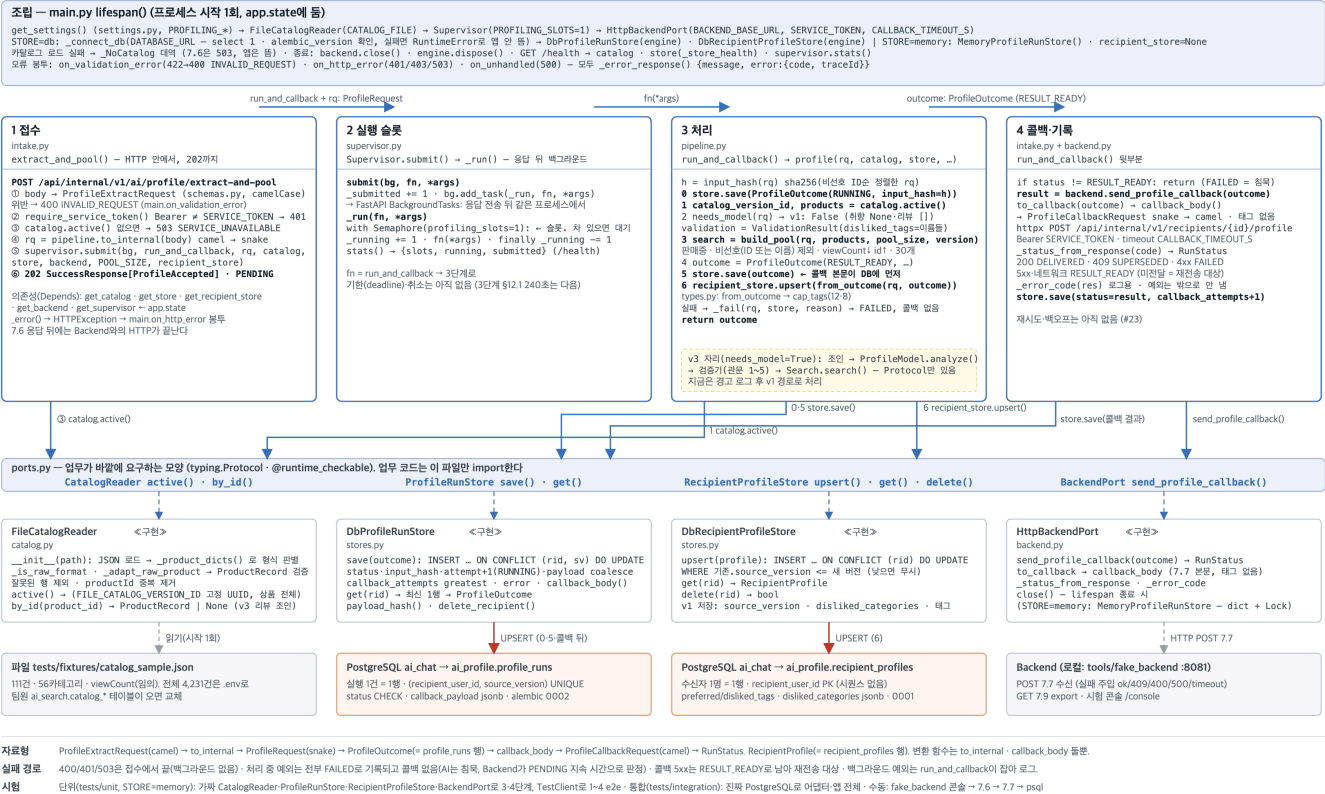


파이프라인 지도 — 2026-09-23 (구조 평탄화: 6폴더 → 1폴더)

기준 코드	브랜치 <code>integration/v1-profiling-search</code> · 커밋 <code>d2c3f41</code>
범위	v1 — 비선택 카테고리만 반영. 취향 문장-리뷰를 읽는 모델-검증기는 v3
동작	7.6 접속 → 202 → 백그라운드 → 비선택 제외-조회수순 30개 → 7.7 콜백 → 결과를 DB에 기록. 09-22와 같음 — 동작은 하나도 안 바뀌었다
저장소	PostgreSQL(<code>docker compose</code>) — <code>ai_profile.profile_runs</code> · <code>ai_profile.recipient_profiles</code> . 카탈로그는 아직 파일
테스트	<code>uv run pytest -q</code> → 75 passed, 3 skipped (v3 stub). 평탄화 전과 같은 수
지난 판	2026-09-22.md

프로파일링 파이프라인 지도 — 단계마다 부르는 함수와 연결된 것 (v1, 2026-09-23 코드 기준)

위에서 아래로: 조합 → 요청 한 건의 단계 → 프로토콜(구현) → 아키텍처(구현) → 바깥 · 호출 = 호출 · 별감 = DB에 쓸 · 화제 정션 = 조건부/HTTP · 노랑 정션 = v3 자리(미구현). 숫자 = pipeline.profile()의 단계 번호.



자료형 ProfileExtractRequest(camel) → to_internal → ProfileRequest(snake) → ProfileOutcome(= profile_runs 행) → callback_body → ProfileCallbackRequest(camel) → RunStatus, RecipientProfile(= recipient_profiles 행). 변환 함수는 to_internal · callback_body 둘.

실패 경로 400/401/503은 접속에서 끝(백그라운드 없음) · 처리 중 예외는 전부 FAILED로 기록되고 콜백 없음(A)는 침묵, Backend가 PENDING 지속 시간으로 판정 · 콜백 5xx는 RESULT_READY로 남아 재전송 대상 · 백그라운드 예외는 run_and_callback이 잡아 로깅

시험 단위(tests/unit, STORE=memory): 가짜 CatalogReader-ProfileRunStore-RecipientProfileStore-BackendPort로 3-4단계, TestClient로 1-4 e2e · 통합(tests/integration): 진짜 PostgreSQL로 아키텍처 일 전체 · 수도: fake_backend 콘솔 → 7.6 → 7.7 → rsql

0. 30초 요약 (이 판에서 달라진 것만)

코드가 하는 일은 어제와 같다. 파일이 어디 있는지만 바뀌었다. 팀원의 검색기(`product-search/search/`)가 한 폴더 안에 모듈 네 개로 끝나는 데 비해 우리 쪽은 폴더 6개에 파일 13개 었다. 폴더가 선언하던 계층을 코드가 이미 지키지 않고 있었으므로(`profile/ports.py`가 `transport/schemas`를, `adapters/profile_run_store_db.py`가 `adapters/backend_port_http`를 import했다) 폴더를 없애고 `src/profiling/` 한 폴더에 모듈 11개로 폼다. 경계를 지키는 것은 폴더가 아니라 `ports.py`(Protocol)이고, 그 것은 그대로 남아 있다.

`from profiling.adapters.profile_run_store_db import DbProfileRunStore` → `from profiling.stores import DbProfileRunStore`처럼 `import` 경로만 바뀐다.

1. 단계별 — 지금 부르는 함수

단 계	파일	함수 (하는 일)
조 립 (시 작 1 회)	<code>main.py</code>	<code>lifespan()</code> — 설정 읽고 구현 4개를 만들어 <code>app.state</code> 에 둬. <code>_connect_db()</code> — DB 접속-마이그레이션 버전 확인, 실패면 앱이 안 뜰. 오류 봉 투 핸들러 3개(400/401-503/500). <code>GET /health</code>
① 접 수	<code>intake.py</code>	<code>extract_and_pool()</code> — 본문 검증(틀리면 400) → <code>require_service_token()</code> (401) → 카탈로그 있나(없으면 503) → <code>pipeline.to_internal()</code> → <code>supervisor.submit()</code> → 202 PENDING

단 계	파일	함수 (하는 일)
㉔ 슬 롯	supervisor.py	Supervisor.submit() → _run() — 세마포어(동시 1)를 잡고 run_and_callback 실행. stats() — /health용
㉓ 처 리	pipeline.py	profile() — 0 store.save(RUNNING) → 1 catalog.active() → 2 needs_model() (v1은 False) → 3 build_pool() (판매중·비선호 제외·조회수↓·30) → 4 ProfileOutcome(RESULT_READY) → 5 store.save() → 6 recipient_store.upsert(from_outcome()). 실패는 _fail() → FAILED, 콜백 없음
㉒ 콜 백· 기 록	intake.py · backend.py	run_and_callback() → send_profile_callback(): callback_body() (7.7 본문, 태그 없음) → POST → CallbackResult(200 DELIVERED · 409 SUPERSEDED · 4xx FAILED · 5xx RESULT_READY) → store.save(그 상태, callback_attempts+1)
포 트	ports.py	CatalogReader · ProfileRunStore · RecipientProfileStore · BackendPort — Protocol 4개. v3용 ProfileModel.Embedder 선언은 지웠다 (v3에 들어올 때 다시 추가, §5 9번)
구 현 (여 덱 터)	catalog.py · stores.py · backend.py	FileCatalogReader / DbProfileRunStore·DbRecipientProfileStore·MemoryProfileRunStore·MemoryRecipientProfileStore(목) / HttpBackendPort
자 료 형	types.py · schemas.py	schemas.py = 바깥 계약(camelCase, 7.6·7.7·7.9), types.py = 내부(snake_case) + RecipientProfile 행과 갱신 규칙 (from_outcome.should_replace_cap_tags)
공 개 표 면	__init__.py	profile · ProfileRequest · ProfileOutcome · RunStatus · ErrorCode · Settings · get_settings — 다른 모듈은 이것만 쓰면 된다
바 깁		tests/fixtures/catalog_sample.json(111건) · PostgreSQL ai_chat (alembic 0001-0002) · Backend — 로컬은 tools/fake_backend(:8081)

2. 지난 판(09-22)에서 바뀐 것

무엇	왜	어디
패키지 평탄화 — 6폴더 13모듈 → 1폴더 11모듈	팀원 검색기와 같은 모양. 폴더가 선언한 계층을 코드가 지키지 않고 있었음	src/profiling/* 전부 (아래 표)
recipient_profile.py를 types.py에 합침	프로필 행과 그 값의 규칙이 같은 곳에	types.py 뒤쪽 "수신자 프로필" 절
저장소 4파일을 stores.py 하나로	같은 계약의 DB 판·메모리 판이 나란히 보이게. 이름 충돌은 RUNS_TABLE/PROFILES_TABLE로 해소	stores.py
빈 catalog/ 패키지 삭제	.gitkeep과 빈 __init__.py뿐이었음	—
v3 포트 선언 2개 제거	v1에서 아무도 안 씀. v3에 들어올 때 추가	ports.py
__init__.py를 공개 표면으로	팀원 search/__init__.py와 같은 방식	__init__.py
테스트 파일 2개 이름 변경	없어진 모듈 이름을 달고 있었음	test_adapters_file_memory.py→test_catalog_stores.py, test_backend_port_http.py→test_backend.py
(09-22 밤) 통합 브랜치 생성	팀원 검색기와 한 트리에서 맞춰 보려고	integration/v1-profiling-search = main + 프로파일링 + product-search/workbench/

옛 경로 → 새 경로

옛	새
transport/schemas.py	schemas.py
transport/profile_intake.py	intake.py
profile/types.py · profile/recipient_profile.py	types.py
profile/ports.py	ports.py
profile/pipeline.py	pipeline.py
runtime/supervisor.py	supervisor.py
config/settings.py	settings.py

옛	새
adapters/catalog_reader_file.py	catalog.py
adapters/backend_port_http.py	backend.py
adapters/profile_run_store_{db,memory}.py	adapters/recipient_profile_store_{db,memory}.py
	stores.py

숫자: 파일 20개(빈 `__init__.py` 7 포함)·폴더 6개 → 파일 12개·폴더 1개. 전체 1,943줄 → 1,860줄, 실제 코드는 921줄 → 923줄(공개 표면 `__init__.py`만큼만 늘었다 = 로직은 안 건드렸다는 뜻).

3. 돌려 보는 법과 확인 포인트

```
cd profiling && docker compose up -d && uv run alembic upgrade head

uv run uvicorn tools.fake_backend.app:app --port 8081

uv run uvicorn profiling.main:app --port 8000 --reload
```

브라우저 `http://localhost:8081/` → 7.6 보내기 탭에서 비선호 하나 고르고 전송. 확인 포인트는 09-22 §3과 같다. 이 판에서 더 볼 것:

- `uv run pytest -q` → 75 passed, 3 skipped (평탄화 전과 같은 수 — 로직이 안 바뀌었다는 증거)
- `uv run python -c "import profiling; print(profiling.__all__)"` → 공개 표면 7개
- `uv run ruff check src tests tools alembic` → `stores.py`의 C408 1건만(아래 §4)

4. 팀원이 알아야 할 결정·미결

09-22의 미결(7.7 태그 · 7.9 조회수 필드명 · 제외 vs 감점 · 디바운스 · Backend 저장 테이블)은 그대로다 — 09-22 §4 참조. 이 판에서 더해진 것:

항목	상태	내용
패키지 모양	결정 (09-23)	팀원 검색기와 같이 한 폴더 평면. 계층은 폴더가 아니라 <code>ports.py</code> 가 지킨다
상품·카테고리 ID 타입	미결	Backend는 BIGINT. 팀원 검색기는 <code>product_id TEXT·category_id TEXT</code> , 수집본은 <code>KAKAO_GIFT:10002797·CAT-01-02</code> . 양쪽 다 Backend id가 아니다 — Backend 상품 목록으로 재적재해야 함
MemoryRecipientProfileStore	골격만	<code>stores.py</code> 맨 아래. <code>ruff</code> C408 1건은 이 목 코드 — 채울 때 같이 정리
독스트링 분량	미결	코드의 27%가 설계 설명. 내용을 <code>docs/</code> 로 옮기고 코드에는 한 줄만 남기는 안이 있음(§5 10번)

5. 다음 해야 할 일 (인수인계용)

우선순위 순. "끝났다고 보는 기준"이 통과하면 그 항목은 끝.

#	무엇	왜	어디 (파일 · 함수)
1	Backend 미팅 결과 반영 — 7.7 태그, 7.9 조회수 필드명, 제외→감점이면 정렬 키 (비선호 여부, -조회수, id), 비선호 ≤5 잠금	미결 4개가 정해져야 실제 Backend와 붙음	<code>schemas.py(ProfileCallbackRequest·ProductRecord.viewCount·dislikedCategories max_length)·pipeline.build_pool()·tools/fake_backend</code>
2	상품 ID를 Backend 기준으로 재적재	7.7로 보내는 30개가 Backend에 없는 번호면 화면에 아무것도 안 뜬다. 지금은 우리도 팀원도 수집처 ID를 쓰는 중	Backend 상품 목록(파일 또는 7.9) → <code>tools/catalog/fetch_export.py --compare-db</code> → <code>catalog.py</code> 재적재 · 팀원 쪽 <code>workbench/dylan/schema.sql(TEXT→BIGINT)·product-search/search/models.py(list[str]→list[int])·tools/prepare_catalog.py</code>
3	실제 Backend와 첫 연동	로컬 fake만 통과한 상태	<code>.env</code> 의 <code>PROFILING_BACKEND_BASE_URL·SERVICE_TOKEN</code> · Backend의 7.6 호출 시점(디바운스)·7.7 수신 확인
4	접수 단계 중복 판정	설계 §10.2: 같은(수신자, 버전)이 RUNNING이면 새 분석 없음, 결과 있으면 재전송만. 지금은 다시 돌림(<code>attempt+1</code>)	<code>intake.extract_and_pool()</code> 앞부분 — <code>store.get()</code> 으로 상태 보고 분기. 필요하면 <code>ProfileRunStore.get_run(rid, sv)</code> 추가

#	무엇	왜	어디 (파일 · 함수)
5	배포 구성 — AI 앱 컨테이너를 compose에, 환경변수·서비스 토큰·이미지 빌드	09/30 배포	<code>docker-compose.yml</code> 에 <code>ai-app</code> · <code>Dockerfile</code> 신설 · <code>alembic upgrade head</code> 를 시작 명령에
6	<code>MemoryRecipientProfileStore</code> 채우기	STORE=memory에서 프로필 저장을 건너뛰고 있음	<code>stores.py</code> 맨 아래 — <code>upsert(should_replace 규칙)</code> · <code>get</code> · <code>delete</code> · <code>main.py</code> memory 분기 한 줄
7	콜백 재시도(5xx 최대 3회)·재시작 복구(RUNNING→FAILED 정리)	설계 §10.2. 지금은 1회 전송, 5xx면 RESULT_READY로 남기만 함	<code>backend.send_profile_callback()</code> 안 재시도 · <code>main.lifespan</code> 시작 시 RUNNING 행 정리
8	팀원 카탈로그 DB로 교체	지금은 파일. 팀원 <code>ai_search.*</code> 가 생기면	<code>catalog.py</code> 에 DB 구현 추가(<code>CatalogReader</code>) · <code>main.lifespan</code> 한 줄
9	v3 — 마이그레이션 0003 · 검증기 이식(실험 <code>2_validate.py</code>) · <code>ProfileModel</code> 구현(규칙 추출기 1안, Ollama 2안) · Search 호출	취향·리뷰 반영	<code>alembic/versions/0003_*.py</code> · <code>pipeline.profile()</code> 2단계 · <code>model.py</code> 신설 · <code>ports.py</code> 에 <code>ProfileModel.Embedder</code> 다시 추가 · <code>types.merge_explicit_dislikes.to_search_hint</code>
10	독스트링의 설계 근거를 <code>docs/</code> 로 이관	코드의 27%가 설명문(팀원 코드는 1%). 읽는 사람은 <code>코드_안내서.md</code> 를 보면 됨	<code>ports.py</code> · <code>stores.py</code> · <code>pipeline.py</code> 의 문단형 독스트링 → 한 줄로, 내용은 <code>docs/코드_안내서.md</code>
11	동시성 — Supervisor 세마포어가 anyio 스레드풀을 붙잡는 문제	동시 100건은 통과했으나(0.64초) 슬롯 대기 중 스레드를 점유. 전용 워커 스레드+큐 / 비동기 의존성 / 큐 상한 +503 3안	<code>supervisor.py</code> · <code>intake.py</code>

막히면 볼 것: [README.md](#)(구조·실행·테스트) · [코드_안내서.md](#)(파일·함수별 역할) · [DB_전환_설명.md](#)(테이블·저장 시점) · [이름_대조표.md](#)(같은 뜻 다른 이름) · [BE_연동_필드표.md](#)(BE 전달본) · 설계 근거는 [KTB4_12team/AI/latest/01_모델_API_설계/담당파트_상세설계서_7.6_7.9.md](#)와 위키 서비스-아키텍처-구현-상세 §10·§16.

연락 없이 이어받을 때의 첫 30분: 위 §3을 그대로 실행해 콘솔에서 30개가 오는지 확인 → `uv run pytest -q` → §5의 1번부터.